







# BENEMÉRITA UNIVERSIDAD DE GUADALAJARA

## CENTRO UNIVERSITARIO DE CIENCIAS EXACTAS E INGENIERÍAS

*Departamento de Ciencias Computacionales*

# Programa 1. Procesamiento por Lotes

*2° Actividad*

## Sistemas Operativos

Código de la materia: IL366

Sección: D09

## P R E S E N T A(N)

ICOM - Velázquez Vazquéz Valeria - 224785718

## PROFESOR(A)

Violeta del Rocío Becerra Velázquez

Guadalajara, Jalisco, Mx.

6 de septiembre del 2026

# ÍNDICE

*Índice* ..... 2
*Objetivo de la actividad* ..... 3
*Notas acerca del lenguaje y herramientas* ..... 3
*Estructuras de datos y Tipos de Datos Abstractos (TDA)* ..... 3
*Conclusiones* ..... 4

## OBJETIVO DE LA ACTIVIDAD

Se desarrolló un sistema capaz de simular el procesamiento por lotes de un conjunto de *N* procesos definidos por el usuario, restringiendo la capacidad máxima de ejecución simultánea a bloques de 5 procesos. El sistema se diseñó para capturar y validar los parámetros de cada proceso de manera secuencial, para posteriormente ejecutarlos mediante una política de planificación de Primero en Llegar, Primero en Ser Atendido (FIFO). Durante la simulación, se implementó una interfaz de monitoreo en tiempo real que expone el estado de los lotes pendientes, el progreso del lote en ejecución, el detalle exhaustivo del proceso activo y el historial de procesos terminados, manteniendo un registro de reloj global ininterrumpido hasta la finalización total de las tareas.

## NOTAS ACERCA DEL LENGUAJE Y HERRAMIENTAS

Para el desarrollo de la simulación se empleó el lenguaje **C++**. La elección de este lenguaje se fundamenta en su capacidad para gestionar de manera precisa los recursos del sistema y su soporte nativo para el paradigma de Programación Orientada a Objetos (POO), lo cual resulta indispensable para modelar entidades independientes como los procesos del sistema operativo. Todo el ciclo de codificación, integración y depuración de la arquitectura modular se llevó a cabo utilizando el entorno de desarrollo VS Code.

Un aspecto técnico destacable de esta implementación es la independencia del sistema operativo anfitrión. Se abstrajeron las directivas específicas del sistema (como las pausas de tiempo y la limpieza de búfer visual) mediante un módulo independiente, garantizando que el código fuente sea portable y ejecutable tanto en entornos Windows como en distribuciones basadas en Unix.

## ESTRUCTURAS DE DATOS Y TIPOS DE DATOS ABSTRACTOS (TDA)

El sistema se construyó sobre una arquitectura basada en componentes, utilizando los siguientes Tipos de Datos Abstractos:

**TDA Process:** Se encapsuló la entidad del proceso computacional, resguardando atributos vitales como el identificador único (ID), tiempo máximo estimado (TME), operación aritmética a realizar y los datos de los operandos. Se implementaron métodos de acceso (getters/setters) para garantizar la integridad de la información.

**TDA BatchSimulator:** Se diseñó una clase orquestadora responsable de administrar el ciclo de vida de la simulación. Este componente segmenta el arreglo lineal de procesos en bloques de tamaño fijo (lotes de 5) y administra el reloj global.

**Estructura CompletionEntry:** Se implementó un registro polimórfico ligero capaz de almacenar tanto los datos resultantes de un proceso finalizado como las marcas visuales de separación de lotes (ej. "---- INICIO LOTE 1 ----").

**Arreglos Estáticos:** Se definió un almacenamiento de tamaño fijo (hasta 50 procesos) para alojar la cola de trabajos, evitando la fragmentación de memoria dinámica y garantizando tiempos de acceso constantes durante la ejecución.

### SOLUCIÓN Y FLUJO DE EJECUCIÓN

La resolución del problema se abordó mediante la separación estricta de responsabilidades (Separation of Concerns), dividiendo el flujo en las siguientes fases:

**Fase de Captura y Validación:** Se delegó la interacción del usuario al módulo InputHelper. A través de este, se capturan secuencialmente los datos, asegurando en tiempo de inserción que los identificadores de programa sean irrepetibles, que el Tiempo Máximo Estimado sea superior a cero y que se eviten indeterminaciones matemáticas (como la división entre cero).

**Fase de Orquestación de Lotes:** Una vez conformada la cola de procesos, el control se transfiere al BatchSimulator. Se calcula matemáticamente la cantidad total de lotes requeridos utilizando una división con techo (ceil).

**Fase de Ejecución y Renderizado:** Se itera sobre cada proceso del lote activo. Por cada unidad de tiempo (segundo), se invoca a la capa de abstracción Platform::sleepOneSecond() para suspender el hilo principal, simulando el costo computacional. Simultáneamente, se actualiza el contador global y se redibuja la consola de manera dinámica, mostrando los tiempos restantes y transcurridos de manera visual.

**Fase de Registro Histórico:** Al concluir el tiempo de un proceso, se calcula su resultado aritmético y se consolida un registro inmutable en el historial de finalizados, actualizando los indicadores de pantalla antes de despachar el siguiente trabajo en espera.

## CONCLUSIONES

A través de la implementación de esta simulación, se logró visualizar de manera tangible el comportamiento de los primeros esquemas de planificación de los sistemas operativos. El procesamiento por lotes demostró ser un mecanismo ordenado, predecible y altamente estructurado para el despacho de tareas secuenciales.

Desde una perspectiva de ingeniería de software, el haber estructurado el sistema dividiendo la interfaz gráfica, la validación de entradas y la lógica de simulación en módulos independientes, demostró que la mantenibilidad y escalabilidad del código aumentan considerablemente. Esta práctica no solo resolvió los requerimientos técnicos de la actividad, sino que sentó las bases arquitectónicas para futuras implementaciones donde se requieran algoritmos de planificación más complejos, como *Round Robin* o colas multinivel.